2. If $B$ is deleted, we get the set $\{A \rightarrow C, B \rightarrow AC$, and $C \rightarrow AB\}$. This case is symmetrical to the previous case, leading to the canonical covers

$$F_c = \{A \rightarrow C, C \rightarrow B, \text{ and } B \rightarrow A\}, \text{ and}$$
$$F_c = \{A \rightarrow C, B \rightarrow C, \text{ and } C \rightarrow AB\}.$$

As an exercise, can you find one more canonical cover for $F$?

## 7.4 Decomposition

The bad design of Section 7.2 suggests that we should *decompose* a relation schema that has many attributes into several schemas with fewer attributes. Careless decomposition, however, may lead to another form of bad design.

Consider an alternative design in which we decompose *Lending-schema* into the following two schemas:

*Branch-customer-schema* = (*branch-name, branch-city, assets, customer-name*)
*Customer-loan-schema* = (*customer-name, loan-number, amount*)

Using the *lending* relation of Figure 7.1, we construct our new relations *branch-customer* (*Branch-customer*) and *customer-loan* (*Customer-loan-schema*):

$$branch\text{-}customer = \Pi_{branch\text{-}name,\ branch\text{-}city,\ assets,\ customer\text{-}name}\ (lending)$$
$$customer\text{-}loan = \Pi_{customer\text{-}name,\ loan\text{-}number,\ amount}\ (lending)$$

Figures 7.9 and 7.10, respectively, show the resulting *branch-customer* and *customer-name* relations.

Of course, there are cases in which we need to reconstruct the *loan* relation. For example, suppose that we wish to find all branches that have loans with amounts less than $1000. No relation in our alternative database contains these data. We need to reconstruct the *lending* relation. It appears that we can do so by writing

$$branch\text{-}customer \bowtie customer\text{-}loan$$

| *branch-name* | *branch-city* | *assets* | *customer-name* |
|---|---|---|---|
| Downtown | Brooklyn | 9000000 | Jones |
| Redwood | Palo Alto | 2100000 | Smith |
| Perryridge | Horseneck | 1700000 | Hayes |
| Downtown | Brooklyn | 9000000 | Jackson |
| Mianus | Horseneck | 400000 | Jones |
| Round Hill | Horseneck | 8000000 | Turner |
| Pownal | Bennington | 300000 | Williams |
| North Town | Rye | 3700000 | Hayes |
| Downtown | Brooklyn | 9000000 | Johnson |
| Perryridge | Horseneck | 1700000 | Glenn |
| Brighton | Brooklyn | 7100000 | Brooks |

**Figure 7.9** The relation *branch-customer*.

| *customer-name* | *loan-number* | *amount* |
|---|---|---|
| Jones | L-17 | 1000 |
| Smith | L-23 | 2000 |
| Hayes | L-15 | 1500 |
| Jackson | L-14 | 1500 |
| Jones | L-93 | 500 |
| Turner | L-11 | 900 |
| Williams | L-29 | 1200 |
| Hayes | L-16 | 1300 |
| Johnson | L-18 | 2000 |
| Glenn | L-25 | 2500 |
| Brooks | L-10 | 2200 |

**Figure 7.10** The relation *customer-loan*.

Figure 7.11 shows the result of computing $branch\text{-}customer \bowtie customer\text{-}loan$. When we compare this relation and the *lending* relation with which we started (Figure 7.1), we notice a difference: Although every tuple that appears in the *lending* relation appears in $branch\text{-}customer \bowtie customer\text{-}loan$, there are tuples in $branch\text{-}customer \bowtie customer\text{-}loan$ that are not in *lending*. In our example, *branch-customer* ⋈ *customer-loan* has the following additional tuples:

(Downtown, Brooklyn, 9000000, Jones, L-93, 500)
(Perryridge, Horseneck, 1700000, Hayes, L-16, 1300)
(Mianus, Horseneck, 400000, Jones, L-17, 1000)
(North Town, Rye, 3700000, Hayes, L-15, 1500)

Consider the query, "Find all bank branches that have made a loan in an amount less than $1000." If we look back at Figure 7.1, we see that the only branches with loan amounts less than $1000 are Mianus and Round Hill. However, when we apply the expression

$$\Pi_{branch\text{-}name} (\sigma_{amount < 1000} (branch\text{-}customer \bowtie customer\text{-}loan))$$

we obtain *three* branch names: Mianus, Round Hill, and Downtown.

A closer examination of this example shows why. If a customer happens to have several loans from different branches, we cannot tell which loan belongs to which branch. Thus, when we join *branch-customer* and *customer-loan*, we obtain not only the tuples we had originally in *lending*, but also several additional tuples. Although we have *more* tuples in $branch\text{-}customer \bowtie customer\text{-}loan$, we actually have *less* information. We are no longer able, in general, to represent in the database information about which customers are borrowers from which branch. Because of this loss of information, we call the decomposition of *Lending-schema* into *Branch-customer-schema* and *customer-loan-schema* a **lossy decomposition**, or a **lossy-join decomposition**. A decomposition that is not a lossy-join decomposition is a **lossless-join decomposi-**

| *branch-name* | *branch-city* | *assets* | *customer-name* | *loan-number* | *amount* |
|---|---|---|---|---|---|
| Downtown | Brooklyn | 9000000 | Jones | L-17 | 1000 |
| Downtown | Brooklyn | 9000000 | Jones | L-93 | 500 |
| Redwood | Palo Alto | 2100000 | Smith | L-23 | 2000 |
| Perryridge | Horseneck | 1700000 | Hayes | L-15 | 1500 |
| Perryridge | Horseneck | 1700000 | Hayes | L-16 | 1300 |
| Downtown | Brooklyn | 9000000 | Jackson | L-14 | 1500 |
| Mianus | Horseneck | 400000 | Jones | L-17 | 1000 |
| Mianus | Horseneck | 400000 | Jones | L-93 | 500 |
| Round Hill | Horseneck | 8000000 | Turner | L-11 | 900 |
| Pownal | Bennington | 300000 | Williams | L-29 | 1200 |
| North Town | Rye | 3700000 | Hayes | L-15 | 1500 |
| North Town | Rye | 3700000 | Hayes | L-16 | 1300 |
| Downtown | Brooklyn | 9000000 | Johnson | L-18 | 2000 |
| Perryridge | Horseneck | 1700000 | Glenn | L-25 | 2500 |
| Brighton | Brooklyn | 7100000 | Brooks | L-10 | 2200 |

**Figure 7.11** The relation $branch\text{-}customer \bowtie customer\text{-}loan$.

**tion**. It should be clear from our example that a lossy-join decomposition is, in general, a bad database design.

Why is the decomposition lossy? There is one attribute in common between *Branch-customer-schema* and *Customer-loan-schema*:

$$Branch\text{-}customer\text{-}schema \cap Customer\text{-}loan\text{-}schema = \{customer\text{-}name\}$$

The only way that we can represent a relationship between, for example, *loan-number* and *branch-name* is through *customer-name*. This representation is not adequate because a customer may have several loans, yet these loans are not necessarily obtained from the same branch.

Let us consider another alternative design, in which we decompose *Lending-schema* into the following two schemas:

$$Branch\text{-}schema = (branch\text{-}name, branch\text{-}city, assets)$$
$$Loan\text{-}info\text{-}schema = (branch\text{-}name, customer\text{-}name, loan\text{-}number, amount)$$

There is one attribute in common between these two schemas:

$$Branch\text{-}loan\text{-}schema \cap Customer\text{-}loan\text{-}schema = \{branch\text{-}name\}$$

Thus, the only way that we can represent a relationship between, for example, *customer-name* and *assets* is through *branch-name*. The difference between this example and the preceding one is that the assets of a branch are the same, regardless of the customer to which we are referring, whereas the lending branch associated with a certain loan amount *does* depend on the customer to which we are referring. For a given *branch-name*, there is exactly one *assets* value and exactly one *branch-city*;

whereas a similar statement cannot be made for *customer-name*. That is, the functional dependency

$$\textit{branch-name} \rightarrow \textit{assets branch-city}$$

holds, but *customer-name* does not functionally determine *loan-number*.

The notion of lossless joins is central to much of relational-database design. Therefore, we restate the preceding examples more concisely and more formally. Let $R$ be a relation schema. A set of relation schemas $\{R_1, R_2, \ldots, R_n\}$ is a **decomposition** of $R$ if

$$R = R_1 \cup R_2 \cup \cdots \cup R_n$$

That is, $\{R_1, R_2, \ldots, R_n\}$ is a decomposition of $R$ if, for $i = 1, 2, \ldots, n$, each $R_i$ is a subset of $R$, and every attribute in $R$ appears in at least one $R_i$.

Let $r$ be a relation on schema $R$, and let $r_i = \Pi_{R_i}(r)$ for $i = 1, 2, \ldots, n$. That is, $\{r_1, r_2, \ldots, r_n\}$ is the database that results from decomposing $R$ into $\{R_1, R_2, \ldots, R_n\}$. It is always the case that

$$r \subseteq r_1 \bowtie r_2 \bowtie \cdots \bowtie r_n$$

To see that this assertion is true, consider a tuple $t$ in relation $r$. When we compute the relations $r_1, r_2, \ldots, r_n$, the tuple $t$ gives rise to one tuple $t_i$ in each $r_i$, $i = 1, 2, \ldots, n$. These $n$ tuples combine to regenerate $t$ when we compute $r_1 \bowtie r_2 \bowtie \cdots \bowtie r_n$. The details are left for you to complete as an exercise. Therefore, every tuple in $r$ appears in $r_1 \bowtie r_2 \bowtie \cdots \bowtie r_n$.

In general, $r \neq r_1 \bowtie r_2 \bowtie \cdots \bowtie r_n$. As an illustration, consider our earlier example, in which

- $n = 2$.
- $R =$ *Lending-schema*.
- $R_1 =$ *Branch-customer-schema*.
- $R_2 =$ *Customer-loan-schema*.
- $r =$ the relation shown in Figure 7.1.
- $r_1 =$ the relation shown in Figure 7.9.
- $r_2 =$ the relation shown in Figure 7.10.
- $r_1 \bowtie r_2 =$ the relation shown in Figure 7.11.

Note that the relations in Figures 7.1 and 7.11 are not the same.

To have a lossless-join decomposition, we need to impose constraints on the set of possible relations. We found that the decomposition of *Lending-schema* into *Branch-schema* and *Loan-info-schema* is lossless because the functional dependency

$$\textit{branch-name} \rightarrow \textit{branch-city assets}$$

holds on *Branch-schema*.